TEMA 1

Fundamentos de la programación

Del problema al algoritmo y del algoritmo al programa

Propósito del tema

Comprender los conceptos, aplicarlos en Java y ser capaz de explicar las decisiones tomadas.

PROGRAMACIÓN · 1.º DAM

Qué aprenderás

- Distinguir problema, algoritmo, programa, código fuente y datos.
- Comprender cómo se traducen y ejecutan los lenguajes de programación.
- Representar soluciones mediante pseudocódigo y diagramas de flujo.
- Aplicar un proceso sistemático de análisis, diseño, prueba y mejora.

> **Ruta de aprendizaje**
> Conceptos → ejemplos resueltos → práctica guiada → ejercicios de consolidación.

Índice del tema

1. Programar es resolver problemas
2. Lenguajes y niveles de representación
3. Compiladores, intérpretes y máquinas virtuales
4. Paradigmas de programación
5. Pseudocódigo
6. Diagramas de flujo
7. Proceso de desarrollo
8. Errores y calidad

1. Programar es resolver problemas

Programar consiste en describir, mediante instrucciones precisas, cómo transformar unos datos de entrada en resultados útiles. El código es la representación final de una solución que antes debe comprenderse y diseñarse.

Concepto	Significado	Ejemplo
Problema	Necesidad que debe resolverse	Calcular el total de una compra
Algoritmo	Secuencia finita y ordenada de pasos	Leer precio, calcular IVA, mostrar total
Programa	Algoritmo expresado en un lenguaje ejecutable	Aplicación Java de consola
Dato	Representación de información	precio = 12.50

Criterios de un buen algoritmo

Debe ser finito, preciso, comprensible, reproducible y producir una salida a partir de unas entradas.

Actividad inicial · La máquina expendedora

Identifica entradas, reglas, salidas y posibles errores de una máquina que vende productos y devuelve cambio.

2. Lenguajes y niveles de representación

El procesador ejecuta instrucciones en lenguaje máquina. Para que las personas puedan programar con mayor productividad se utilizan lenguajes ensambladores y lenguajes de alto nivel.

Nivel	Características	Ventaja principal
Máquina	Bits e instrucciones específicas del procesador	Ejecución directa
Ensamblador	Mnemónicos muy próximos al hardware	Control detallado
Alto nivel	Abstracciones, tipos y estructuras legibles	Productividad y portabilidad

Los lenguajes de alto nivel reducen el esfuerzo de desarrollo y mantenimiento, aunque introducen una capa de traducción y abstracción respecto al hardware.

3. Compiladores, intérpretes y máquinas virtuales

Un traductor convierte el código fuente a una representación que la máquina pueda ejecutar. La compilación traduce el programa antes de ejecutarlo; la interpretación procesa instrucciones durante la ejecución. Muchos entornos modernos combinan ambos enfoques.

Enfoque	Cuándo traduce	Consecuencia
Compilado	Antes de ejecutar	Detecta muchos errores pronto y genera código objeto
Interpretado	Durante la ejecución	Facilita la portabilidad y la experimentación
Máquina virtual	Compila a código intermedio y lo ejecuta	Equilibrio entre portabilidad y rendimiento

> **Caso Java**
>
> El compilador javac genera bytecode (.class). La JVM carga, verifica y ejecuta ese bytecode en cada plataforma.

4. Paradigmas de programación

Un paradigma es una forma de organizar el razonamiento y el código. Los lenguajes pueden admitir varios paradigmas.

- Imperativo: describe una secuencia de cambios de estado.
- Estructurado: organiza la solución mediante secuencia, selección y repetición.
- Funcional: compone funciones y reduce los cambios de estado.
- Orientado a objetos: modela responsabilidades mediante objetos que colaboran.

En estos primeros temas trabajaremos principalmente programación estructurada. Las clases y los objetos se abordarán después.

5. Pseudocódigo

El pseudocódigo permite expresar la lógica sin depender de la sintaxis exacta de Java. Debe emplear verbos claros, indentación y estructuras de control reconocibles.

```
INICIO
    LEER precio
    LEER unidades
    subtotal <- precio * unidades
    total <- subtotal * 1.21
    ESCRIBIR total
FIN
```

> **Ventaja didáctica**
>
> Separar lógica y sintaxis ayuda a detectar errores de planteamiento antes de programar.

Práctica guiada · Conversor de tiempo

Escribe el pseudocódigo que transforma una cantidad total de segundos en horas, minutos y segundos.

6. Diagramas de flujo

Un diagrama de flujo representa visualmente el orden de las operaciones. Los símbolos básicos son: óvalo para inicio y fin, rectángulo para proceso, paralelogramo para entrada o salida, rombo para decisión y flechas para el flujo.

Símbolo	Uso	Pregunta útil
Inicio/fin	Delimita el algoritmo	¿Dónde comienza y termina?
Proceso	Cálculo o asignación	¿Qué valor cambia?
Entrada/salida	Lectura o presentación	¿Qué recibe o muestra?
Decisión	Bifurcación según condición	¿Qué ocurre si es verdadero o falso?

Buena práctica

Evita cruces innecesarios, usa un único sentido de lectura y etiqueta claramente las ramas de cada decisión.

7. Proceso de desarrollo

Resolver un problema de forma profesional implica repetir un ciclo: analizar requisitos, diseñar el algoritmo, codificar, compilar, probar, depurar, documentar y mantener.

- Analizar: identificar entradas, salidas, reglas y casos límite.
- Diseñar: descomponer el problema en pasos pequeños.
- Implementar: traducir el diseño al lenguaje elegido.
- Verificar: comparar resultados reales con resultados esperados.
- Mejorar: simplificar, renombrar, documentar y eliminar duplicación.

Consolidación · Plan de pruebas

Para un programa que calcula el precio final con descuento, define tres casos normales, dos casos límite y una entrada inválida.

8. Errores y calidad

Tipo de error	Cuándo aparece	Cómo actuar
Sintaxis/compilación	El traductor no puede generar el programa	Leer el primer mensaje y revisar la línea indicada
Ejecución	El programa comienza pero falla	Reproducir el caso y revisar datos y excepciones
Lógico	El programa termina con resultado incorrecto	Trazar valores y contrastar con ejemplos calculados

Depurar no es adivinar

Formula una hipótesis, observa valores, realiza un cambio pequeño y vuelve a comprobar.

Síntesis y autoevaluación

- El algoritmo es la solución; el programa es su implementación.
- Los lenguajes de alto nivel necesitan traducción para ejecutarse.
- Pseudocódigo y diagramas ayudan a diseñar antes de codificar.
- La calidad se construye mediante pruebas, depuración y revisión.

Reto final · Comprueba que dominas el tema

Diseña en pseudocódigo un programa que lea diez números, calcule su media y muestre el mayor. Añade una tabla de casos de prueba y explica qué errores podrían aparecer al implementarlo.

Material base

Tema reorganizado y desarrollado a partir de programación-fundamentos-de-la-programación.pdf, aportado por el docente.